

✓ Authenticate Google

```
#This code authenticates your Google account in Google Colab and gives your Py
from google.colab import auth
from google.auth import default
import gspread

# Authenticate to Colab
# This will prompt you to open a new browser window to authenticate your Google
auth.authenticate_user()

# Authorize gspread with your Google credentials
# ===== SET AUTH PARAMETER =====
creds, _ = default()
gc = gspread.authorize(creds)
```

```
import google.auth
# Get the authenticated credentials
credentials, _ = google.auth.default()
```

✓ Functions

```
import pandas as pd
#This function loads data from a Google Sheet and converts it into a Pandas Data

def get_sheet(sheet_name, worksheet_name = 'Data'):
    # Open the spreadsheet by name
    spreadsheet = gc.open(sheet_name)

    # Select a worksheet by name or index
    worksheet = spreadsheet.worksheet(worksheet_name)

    # Get all values from the worksheet as a list of lists
    data = worksheet.get_all_values()

    # Convert to a Pandas DataFrame
    df_sheet = pd.DataFrame(data[1:], columns=data[0])

    print("Google Sheet loaded successfully into DataFrame 'df_sheet'.")

    return df_sheet
```

```
# @title
#This function writes the contents of a Pandas DataFrame to a Google Sheet, sta
def add_extraction_to_sheet(sheet_name, data_to_fill, gsheet_name=None, gsheets
    """
```

```

Append one or more rows of extracted data to a worksheet, writing values
starting at `start_col` (default "A") through to the end of the row

sheet_name    : name of the worksheet tab to write to
data_to_fill  : pandas DataFrame with the row(s) to append
                (column order should match the sheet's header row, starting at
gsheet_name   : name of the Google Sheet to open (used if gsheet not given)
gsheet        : an already-opened gspread Spreadsheet object
start_col     : column letter to start writing from (default "A")
"""

if gsheet is None and gsheet_name is None:
    raise ValueError("Either gsheet or gsheet_name must be provided.")

gsheets = gsheet if gsheet is not None else gc.open(gsheet_name)
sheet = gsheets.worksheet(sheet_name)

# Replace NaN/NaT with None so gspread doesn't choke on them
clean_df = data_to_fill.astype(object).where(pd.notnull(data_to_fill), None)

rows = clean_df.values.tolist()
data_range = f"{start_col}{index + 1}"

return sheet.update(range_name=data_range, values=rows)

```

```

#This function writes the contents of a Pandas DataFrame to a Google Sheet, starting at
def add_extraction_to_sheet(sheet_name, data_to_fill, gsheet_name=None, gsheets=None):
    """
    Append one or more rows of extracted data to a worksheet, writing values
    starting at `start_col` (default "A") through to the end of the row

    sheet_name    : name of the worksheet tab to write to
    data_to_fill  : pandas DataFrame with the row(s) to append
                    (column order should match the sheet's header row, starting at
    gsheet_name   : name of the Google Sheet to open (used if gsheet not given)
    gsheet        : an already-opened gspread Spreadsheet object
    start_col     : column letter to start writing from (default "A")
    """

    if gsheet is None and gsheet_name is None:
        raise ValueError("Either gsheet or gsheet_name must be provided.")

    gsheets = gsheet if gsheet is not None else gc.open(gsheet_name)
    sheet = gsheets.worksheet(sheet_name)

    # Replace NaN/NaT with None so gspread doesn't choke on them
    clean_df = data_to_fill.astype(object).where(pd.notnull(data_to_fill), None)

    rows = clean_df.values.tolist()
    sheet.append_rows(rows)
    return True

```

```

#It sends an image to an AI model (like GPT-4 Vision, a local VLM, Groq-hosted

import os
from openai import OpenAI

def extract_text(

```

```

    image_url: str,
    prompt: str,
    model: str,
    base_url: str,
    api_key: str = None,
    max_tokens: int = 2000,
    temperature: float = 0.0,
) -> str:

    resolved_base_url = base_url
    resolved_model = model
    resolved_api_key = api_key

    client_kwargs = {}

    if resolved_api_key is not None:
        client_kwargs["api_key"] = resolved_api_key

    if resolved_base_url:
        client_kwargs["base_url"] = resolved_base_url

    client = OpenAI(**client_kwargs)

    response = client.chat.completions.create(
        model=resolved_model,
        messages=[
            {
                "role": "user",
                "content": [
                    {"type": "text", "text": prompt},
                    {"type": "image_url", "image_url": {"url": image_url}},
                ],
            }
        ],
        max_tokens=max_tokens,
        temperature=temperature,
        reasoning_effort="none",
        response_format={"type": "json_object"},
    )

    return response.choices[0].message.content.strip()

```

✓ Leverage

```

# Configure Google Sheet inputs and LLM settings for receipt data extraction.
# Defines the source worksheet containing receipt URLs, the annotation output sheet,
# the AI model, API endpoint, and authentication key required to call the LLM.

# Name of the worksheet that contains the main dataset.
receipt_url_sheet = "DN_XT_026_data"

# Name of the annotation template
annotation_sheet="DN_XT_026_Annotation_Template"

from google.colab import userdata

```

```
# LLM model identifier (e.g. "qwen2.5:7b", "gpt-4.1-mini").
model = "qwen/qwen3.6-27b"

# Base URL of the LLM provider or API endpoint.
# Example: "https://api.openai.com/v1"
base_url = "https://api.groq.com/openai/v1"

# API key for the provider.
# Set to None if the endpoint does not require authentication.
api_key = userdata.get('GROQ_API_KEY')
```

```
#load the Google Sheet data from the worksheet specified in receipt_url_sheet
main_df = get_sheet(receipt_url_sheet)
```

Google Sheet loaded successfully into DataFrame 'df_sheet'.

```
# YOU CAN UPDATE OR OPTIMIZE THESE PROMPTS
image_url = main_df.iloc[0, 1]
reciept_id = main_df.iloc[0, 0]

RECIEPTS_DETAIL_PROMPT = (
    "You are a receipt data extraction engine. Extract receipt data from the image and return ONLY a single valid JSON object with this schema: "
    "{receipt_id:null, receipt_number, receipt_date, receipt_time, subtotal, "
    "tax_amount, discount_amount, total_amount, currency, payment_method, "
    "image_url:null, ocr_raw_text, extraction_model:null, status:null, "
    "savings_amount, loyalty_points_earned, loyalty_points_redeemed, "
    "loyalty_card_number, loyalty_program, store_name, store_location}. "
    "Use null for missing or unreadable fields. Never guess values. "
    "Normalize dates to YYYY-MM-DD, times to HH:MM (24-hour), monetary values "
    "to plain numbers, currency to a 3-letter ISO code when inferable, and "
    "loyalty points to integers. Preserve the receipt text exactly as printed "
    "in ocr_raw_text with original line breaks."
)

PRODUCT_DETAIL_PROMPT = (
    'Extract all purchased receipt line items. Return ONLY valid JSON in this structure: {"products": [{raw_text, item_name, quantity, unit_price, discount, total_price, sequence_order, receipt_index}]}. '
    'One object per purchased item in receipt order. '
    'Ignore subtotal, tax, tip, rebates, discount summaries, payment information and total lines. '
    'Use null for any field that is missing or genuinely unreadable. Never invent digits that are not visible. If part of a number is obscured, use null rather than guessing that number. '
    'Do not use the receipt grand total to reconstruct or infer obscured item values. '
    'For crossed-out or handwritten-over text, extract only the final printed value that are clearly visible; use null if the final value cannot be determined. '
    'Keep raw_text as close as possible to the printed item text. '
    'Lightly clean item_name only. '
    'quantity, unit_price, discount, and total_price must be numbers or null. '
    'Do not calculate missing total_price from quantity and unit_price. '
    'sequence_order starts at 1. Always set receipt_index to null.'
)
```

✓ Reciept Detail Prompt

```
#sends a receipt image to the AI vision model for extraction and stores the AI
output = extract_text(
    image_url=image_url,
    prompt=RECIEPTS_DETAIL_PROMPT,
    model=model,
    base_url=base_url,
    api_key=api_key,
    max_tokens=4096, # Further increased max_tokens to allow for complete JSON
    temperature=0.0,
)
```

```
import json
import re
import pandas as pd

# First, clean the output from any <think> blocks
cleaned_output = re.sub(r'<think>.*?</think>', '', output, flags=re.DOTALL)
cleaned_output = cleaned_output.strip()

# Find the start and end markers of the JSON code block
json_start_marker = "`json\n"
json_end_marker = "\n`"

start_index = cleaned_output.find(json_start_marker)
end_index = cleaned_output.rfind(json_end_marker)

json_string = ""
if start_index != -1 and end_index != -1 and start_index < end_index:
    # Extract the substring containing only the JSON
    json_string = cleaned_output[start_index + len(json_start_marker) : end_index]
    data = json.loads(json_string)
elif start_index != -1 and end_index == -1: # If only start marker is found, it's truncated
    json_string = cleaned_output[start_index + len(json_start_marker):] # Take the rest
    try: # Added try-except specifically for this truncated case
        data = json.loads(json_string)
    except json.JSONDecodeError as e:
        raise ValueError(
            f"LLM output truncated: JSON block started but did not end. "
            f"Resulting JSON is incomplete and invalid. Error: {e}. "
            f"Consider increasing 'max_tokens' in the 'extract_text' call. "
            f"Incomplete JSON string (first 200 chars): {json_string[:200]}..."
        )
else:
    # Fallback if the markdown block is not found, try to load the entire cleaned output
    try:
        data = json.loads(cleaned_output)
    except json.JSONDecodeError as e: # Modified to include error message
        # If it still fails, raise a more informative error or handle it
        raise ValueError(f"Could not extract valid JSON from the LLM output. Error: {e}")
```

```

data["receipt_id"] = receipt_id
data["image_url"] = image_url
data["extraction_model"] = model

receipt_data = pd.DataFrame(data, index=[0])

# Rename fields to match the annotation template
receipt_data = receipt_data.rename(columns={
    "store_name": "store name",
    "store_location": "store location"
})

# Arrange columns in the exact order of the annotation template
receipt_columns = [
    "receipt_id",
    "receipt_number",
    "receipt_date",
    "receipt_time",
    "subtotal",
    "tax_amount",
    "discount_amount",
    "total_amount",
    "currency",
    "payment_method",
    "image_url",
    "ocr_raw_text",
    "extraction_model",
    "status",
    "savings_amount",
    "loyalty_points_earned",
    "loyalty_points_redeemed",
    "loyalty_card_number",
    "loyalty_program",
    "store name",
    "store location"
]

receipt_data = receipt_data.reindex(columns=receipt_columns)

receipt_data

```

```

#write receipt details into annotation sheet
add_extraction_to_sheet(
    "Receipt Details",
    receipt_data,
    gsheets_name=annotation_sheet
)

```

True

✓ Product Detail Prompt

```

#sends a receipt image to the AI vision model for extraction and stores the AI
output = extract_text(
    image_url=image_url,
    prompt=PRODUCT_DETAIL_PROMPT,
    model=model,
    base_url=base_url,
    api_key=api_key,
    max_tokens=4000,
    temperature=0.0,
)

```

```

# Process LLM extraction output and prepare structured receipt data.
import json
import re

# Find the start and end markers of the JSON code block
json_start_marker = "```json\n"
json_end_marker = "\n```"

start_index = output.find(json_start_marker)
end_index = output.rfind(json_end_marker)

if start_index != -1 and end_index != -1:
    # Extract the substring containing only the JSON
    json_string = output[start_index + len(json_start_marker) : end_index]
    data = json.loads(json_string)
else:
    # If markdown block is not found, try to clean the output first
    # This removes any '<think>...</think>' blocks from the LLM's response.
    cleaned_output = re.sub(r'<think>.*?</think>', '', output, flags=re.DOTALL)
    cleaned_output = cleaned_output.strip()

    # Then try to load the cleaned output as JSON
    try:
        data = json.loads(cleaned_output)
    except json.JSONDecodeError:
        # If it still fails, raise a more informative error or handle it
        raise ValueError("Could not extract valid JSON from the LLM output even

product_data = pd.DataFrame(data)

product_data["receipt_index"] = receipt_id

product_data = product_data.rename(columns={
    "receipt_index": "Receipt index"
})

# Arrange columns in the exact order of the annotation template
product_columns = [
    "raw_text",
    "item_name",
    "quantity",
    "unit_price",
    "discount",
    "total_price",
    "sequence_order",
    "Receipt index"
]

```

```
product_data = product_data.reindex(columns=product_columns)
```

```
product_data
```

```
#write product details into annotation sheet
add_extraction_to_sheet(
    "Product Details",
    product_data,
    gsheets_name=annotation_sheet
)
```

```
True
```

```
import json
import re

def parse_json_output(output):
    """
    Extract and parse JSON from an LLM response.

    Handles:
    - <think>...</think> blocks
    - ```json ... ``` code blocks
    - plain JSON responses
    """

    if not isinstance(output, str):
        raise ValueError("LLM output is not a string.")

    # 1. Remove <think>...</think> blocks
    cleaned_output = re.sub(
        r"<think>.*?</think>",
        "",
        output,
        flags=re.DOTALL | re.IGNORECASE
    ).strip()

    # 2. Look for a fenced JSON block
    json_match = re.search(
        r"```json\s*(.*?)\s*```",
        cleaned_output,
        flags=re.DOTALL | re.IGNORECASE
    )

    if json_match:
        json_string = json_match.group(1).strip()
        return json.loads(json_string)

    # 3. Look for any fenced code block as a fallback
    code_match = re.search(
        r"```\s*(.*?)\s*```",
        cleaned_output,
        flags=re.DOTALL
```

```

    )

    if code_match:
        json_string = code_match.group(1).strip()
        return json.loads(json_string)

    # 4. Otherwise, assume the remaining response is JSON
    return json.loads(cleaned_output)

```

✓ Run Multiple

This allows you to run multiple in a loop

```

import json
import os

BATCH_SIZE = 28

JSON_FAILURES_FILE = "known_json_failures.json"

# Load previously known JSON failures
if os.path.exists(JSON_FAILURES_FILE):
    with open(JSON_FAILURES_FILE, "r") as f:
        KNOWN_JSON_FAILURES = set(json.load(f))
else:
    KNOWN_JSON_FAILURES = {
        "8a7ed482-acd7-4d48-9791-f24410dc6bb8",
        "7ccd8903-a7d2-4c12-b3c7-e3e42419fc5f",
    }

    # Create the file with the initial known failures
    with open(JSON_FAILURES_FILE, "w") as f:
        json.dump(sorted(KNOWN_JSON_FAILURES), f, indent=2)

print(f"Known JSON failures: {len(KNOWN_JSON_FAILURES)}")

# =====
# OPEN ANNOTATION WORKBOOK
# =====

annotation_workbook = gc.open(annotation_sheet)

# Open the worksheets
receipt_details_ws = annotation_workbook.worksheet("Receipt Details")
product_details_ws = annotation_workbook.worksheet("Product Details")

# =====
# CHECK ALREADY PROCESSED RECEIPTS
# =====

# Get receipt IDs already present in Receipt Details
existing_receipt_ids = set(receipt_details_ws.col_values(1)[1:])

# Get receipt IDs already linked in Product Details
existing_product_receipt_ids = set(product_details_ws.col_values(8)[1:])

```

```

print(f"{len(existing_receipt_ids)} receipt records already exist.")
print(f"{len(existing_product_receipt_ids)} receipts already have product reco

# =====
# FIND RECEIPTS NOT FULLY PROCESSED
# =====

unprocessed_df = main_df[
    ~main_df.iloc[:, 0].isin(
        existing_receipt_ids.intersection(existing_product_receipt_ids)
    )
]

print(f"Receipts remaining to process: {len(unprocessed_df)}")

# =====
# PROCESS BATCH
# =====

for index, row in unprocessed_df.head(BATCH_SIZE).iterrows():

    receipt_id = row.iloc[0]
    image_url = row.iloc[1]

    # -----
    # SKIP KNOWN JSON FAILURES BEFORE ANY API CALL
    # -----

    # Temporarily allow the two previously failed receipts to be retried
    TEST_RETRY_IDS = {
        "8a7ed482-acd7-4d48-9791-f24410dc6bb8",
        "7ccd8903-a7d2-4c12-b3c7-e3e42419fc5f",
    }

    if receipt_id in KNOWN_JSON_FAILURES and receipt_id not in TEST_RETRY_IDS:
        print(
            f"Receipt {receipt_id}: Known JSON failure. "
            "Skipping without API call."
        )
        continue

    try:

        # =====
        # RECEIPT DETAILS
        # =====

        if receipt_id in existing_receipt_ids:

            print(
                f"Receipt {receipt_id}: Receipt Details already exist. "
                "Skipping receipt extraction."
            )

        else:

            print(f"Processing Receipt Details for {receipt_id}...")

```

```

output = extract_text(
    image_url=image_url,
    prompt=RECIEPTS_DETAIL_PROMPT,
    model=model,
    base_url=base_url,
    api_key=api_key,
    max_tokens=2500,
    temperature=0.0,
)

data = parse_json_output(output)

data["receipt_id"] = reciept_id
data["image_url"] = image_url
data["extraction_model"] = model

data = pd.DataFrame(data, index=[0])

# Rename fields to match annotation template
data = data.rename(columns={
    "store_name": "store name",
    "store_location": "store location"
})

# Arrange columns in exact template order
receipt_columns = [
    "receipt_id",
    "receipt_number",
    "receipt_date",
    "receipt_time",
    "subtotal",
    "tax_amount",
    "discount_amount",
    "total_amount",
    "currency",
    "payment_method",
    "image_url",
    "ocr_raw_text",
    "extraction_model",
    "status",
    "savings_amount",
    "loyalty_points_earned",
    "loyalty_points_redeemed",
    "loyalty_card_number",
    "loyalty_program",
    "store name",
    "store location"
]

data = data.reindex(columns=receipt_columns)

add_extraction_to_sheet(
    "Receipt Details",
    data,
    gsheets_name=annotation_sheet
)

existing_receipt_ids.add(reciept_id)

```

```

        print(
            f"Receipt Details successfully written for {reciept_id}."
        )

# =====
# PRODUCT DETAILS
# =====

if reciept_id in existing_product_receipt_ids:

    print(
        f"Receipt {reciept_id}: Product Details already exist. "
        "Skipping product extraction."
    )

else:

    print(f"Processing Product Details for {reciept_id}...")

    output = extract_text(
        image_url=image_url,
        prompt=PRODUCT_DETAIL_PROMPT,
        model=model,
        base_url=base_url,
        api_key=api_key,
        max_tokens=3800,
        temperature=0.0,
    )

    data = parse_json_output(output)
    data = pd.DataFrame(data["products"])

    # Link every product to the receipt
    data["receipt_index"] = reciept_id

    # Rename to match annotation template
    data = data.rename(columns={
        "receipt_index": "Reciept index"
    })

    # Arrange columns in exact template order
    product_columns = [
        "raw_text",
        "item_name",
        "quantity",
        "unit_price",
        "discount",
        "total_price",
        "sequence_order",
        "Reciept index"
    ]

    data = data.reindex(columns=product_columns)

    add_extraction_to_sheet(
        "Product Details",
        data,
        gsheet_name=annotation_sheet
    )

```

```

        existing_product_receipt_ids.add(receipt_id)

    print(
        f"Product Details successfully written for {receipt_id} "
        f"({len(data)} products)."
    )

# =====
# FINAL STATUS
# =====

if (
    receipt_id in existing_receipt_ids
    and receipt_id in existing_product_receipt_ids
):
    print(f"Successfully processed receipt {receipt_id}.")

except Exception as e:
    print(f"FAILED to process receipt {receipt_id}: {e}")

    error_message = str(e)

# =====
# JSON PARSING FAILURE
# =====

if (
    "JSONDecodeError" in error_message
    or "Expecting value" in error_message
    or "Extra data" in error_message
    or "Expecting ',' delimiter" in error_message
    or "Unterminated string" in error_message
):
    print(
        f"Receipt {receipt_id}: JSON parsing failure detected. "
        "Adding to known failures list."
    )

    # Add to in-memory set for the current run
    KNOWN_JSON_FAILURES.add(receipt_id)

    # Save permanently for future runs
    with open(JSON_FAILURES_FILE, "w") as f:
        json.dump(
            sorted(KNOWN_JSON_FAILURES),
            f,
            indent=2
        )

    print(
        f"Receipt {receipt_id} saved to "
        f"{JSON_FAILURES_FILE}."
    )

# =====
# RATE LIMIT
# =====

```

```

        if "429" in error_message or "rate_limit_exceeded" in error_message:
            print(
                "Rate limit reached. Stopping this batch "
                "to avoid further failed API calls."
            )
            break

```

[Show hidden output](#)

```

failed_receipts = failed_receipts_df.iloc[:, [0, 1]].copy()

failed_receipts.columns = ["receipt_id", "image_url"]

print(failed_receipts.to_string(index=False))

```

```

# =====
# AUTOMATED QUALITY CHECK – RECEIPT + PRODUCT EXTRACTION
# =====
# This cell READS the existing annotation sheets only.
# It DOES NOT modify any Google Sheet data.
# =====

import pandas as pd
import numpy as np

# -----
# 1. OPEN ANNOTATION WORKBOOK
# -----

annotation_workbook = gc.open(annotation_sheet)

receipt_ws = annotation_workbook.worksheet("Receipt Details")
product_ws = annotation_workbook.worksheet("Product Details")

# Load sheets into DataFrames
receipt_df = pd.DataFrame(
    receipt_ws.get_all_records()
)

product_df = pd.DataFrame(
    product_ws.get_all_records()
)

print("=" * 70)
print("QUALITY CHECK STARTED")
print("=" * 70)

print(f"\nReceipt records: {len(receipt_df)}")
print(f"Product records: {len(product_df)}")

# -----
# 2. BASIC STRUCTURE CHECK

```

```

# -----

print("\n" + "=" * 70)
print("1. STRUCTURE CHECK")
print("=" * 70)

print("\nReceipt Details columns:")
print(receipt_df.columns.tolist())

print("\nProduct Details columns:")
print(product_df.columns.tolist())

# -----
# 3. RECEIPT ID CHECK
# -----

print("\n" + "=" * 70)
print("2. RECEIPT ID CHECK")
print("=" * 70)

receipt_id_col = "receipt_id"
product_receipt_id_col = "Receipt index"

# Remove empty IDs for checking
receipt_ids = receipt_df[receipt_id_col].astype(str).str.strip()
product_receipt_ids = product_df[product_receipt_id_col].astype(str).str.strip()

# Duplicate receipt IDs
duplicate_receipt_ids = (
    receipt_ids[receipt_ids.duplicated(keep=False)]
    .unique()
    .tolist()
)

# Missing receipt IDs
missing_receipt_ids = receipt_df[
    receipt_df[receipt_id_col].isna()
    | (receipt_df[receipt_id_col].astype(str).str.strip() == "")
]

# Product records with no matching receipt
orphan_products = product_df[
    ~product_receipt_ids.isin(set(receipt_ids))
]

print(f"Unique receipt IDs: {receipt_ids.nunique()}")
print(f"Duplicate receipt IDs: {len(duplicate_receipt_ids)}")
print(f"Missing receipt IDs: {len(missing_receipt_ids)}")
print(f"Products linked to non-existent receipts: {len(orphan_products)}")

if duplicate_receipt_ids:
    print("\nDuplicate receipt IDs:")
    print(duplicate_receipt_ids)

if len(orphan_products) > 0:
    print("\nOrphan product records:")
    print(orphan_products[[product_receipt_id_col]].drop_duplicates())

```

```

# -----
# 4. REQUIRED RECEIPT FIELD CHECK
# -----

print("\n" + "=" * 70)
print("3. RECEIPT FIELD COMPLETENESS")
print("=" * 70)

receipt_required_fields = [
    "receipt_id",
    "receipt_number",
    "receipt_date",
    "receipt_time",
    "total_amount",
    "currency",
    "payment_method",
    "image_url",
    "ocr_raw_text",
]

receipt_missing_report = []

for col in receipt_required_fields:

    if col not in receipt_df.columns:
        receipt_missing_report.append({
            "field": col,
            "missing": "COLUMN NOT FOUND",
            "missing_percentage": None
        })
        continue

    missing = (
        receipt_df[col].isna()
        | (receipt_df[col].astype(str).str.strip() == "")
        | (receipt_df[col].astype(str).str.lower().isin(["nan", "none", "null"])
    )

    receipt_missing_report.append({
        "field": col,
        "missing": int(missing.sum()),
        "missing_percentage": round(missing.mean() * 100, 2)
    })

receipt_missing_report_df = pd.DataFrame(receipt_missing_report)

print(receipt_missing_report_df.to_string(index=False))

# -----
# 5. PRODUCT FIELD COMPLETENESS
# -----

print("\n" + "=" * 70)
print("4. PRODUCT FIELD COMPLETENESS")
print("=" * 70)

product_required_fields = [
    "raw_text",
    "item_name",

```

```

        "quantity",
        "unit_price",
        "discount",
        "total_price",
        "sequence_order",
        "Reciept index",
    ]

    product_missing_report = []

    for col in product_required_fields:

        if col not in product_df.columns:
            product_missing_report.append({
                "field": col,
                "missing": "COLUMN NOT FOUND",
                "missing_percentage": None
            })
            continue

        missing = (
            product_df[col].isna()
            | (product_df[col].astype(str).str.strip() == "")
            | (product_df[col].astype(str).str.lower().isin(["nan", "none", "null"])
        )

        product_missing_report.append({
            "field": col,
            "missing": int(missing.sum()),
            "missing_percentage": round(missing.mean() * 100, 2)
        })

    product_missing_report_df = pd.DataFrame(product_missing_report)

    print(product_missing_report_df.to_string(index=False))

    # -----
    # 6. NUMERIC FIELD CHECK
    # -----

    print("\n" + "=" * 70)
    print("5. NUMERIC FIELD CHECK")
    print("=" * 70)

    numeric_receipt_fields = [
        "subtotal",
        "tax_amount",
        "discount_amount",
        "total_amount",
        "savings_amount",
        "loyalty_points_earned",
        "loyalty_points_redeemed",
    ]

    numeric_product_fields = [
        "quantity",
        "unit_price",
        "discount",
        "total_price",
    ]

```

```

]

receipt_numeric_report = []

for col in numeric_receipt_fields:

    if col not in receipt_df.columns:
        continue

    converted = pd.to_numeric(receipt_df[col], errors="coerce")

    non_numeric = (
        receipt_df[col].notna()
        & (receipt_df[col].astype(str).str.strip() != "")
        & converted.isna()
    )

    receipt_numeric_report.append({
        "field": col,
        "non_numeric_values": int(non_numeric.sum())
    })

print("Receipt numeric fields:")
print(pd.DataFrame(receipt_numeric_report).to_string(index=False))


product_numeric_report = []

for col in numeric_product_fields:

    if col not in product_df.columns:
        continue

    converted = pd.to_numeric(product_df[col], errors="coerce")

    non_numeric = (
        product_df[col].notna()
        & (product_df[col].astype(str).str.strip() != "")
        & converted.isna()
    )

    product_numeric_report.append({
        "field": col,
        "non_numeric_values": int(non_numeric.sum())
    })

print("\nProduct numeric fields:")
print(pd.DataFrame(product_numeric_report).to_string(index=False))


# -----
# 7. PRODUCT MATHEMATICAL CONSISTENCY
# -----

print("\n" + "=" * 70)
print("6. PRODUCT PRICE CONSISTENCY")
print("=" * 70)

product_check_df = product_df.copy()

```

```

for col in ["quantity", "unit_price", "discount", "total_price"]:
    if col in product_check_df.columns:
        product_check_df[col] = pd.to_numeric(
            product_check_df[col],
            errors="coerce"
        )

# Calculate expected amount only where quantity and unit price exist
product_check_df["calculated_amount"] = (
    product_check_df["quantity"]
    * product_check_df["unit_price"]
)

# Difference between calculated amount and extracted total
product_check_df["amount_difference"] = (
    product_check_df["calculated_amount"]
    - product_check_df["total_price"]
).abs()

# Allow small floating-point differences
product_math_issues = product_check_df[
    product_check_df["quantity"].notna()
    & product_check_df["unit_price"].notna()
    & product_check_df["total_price"].notna()
    & (product_check_df["amount_difference"] > 0.02)
]

print(
    f"Products with quantity × unit price different from total price: "
    f"{len(product_math_issues)}"
)

if len(product_math_issues) > 0:
    print("\nThese require MANUAL REVIEW.")
    print(
        product_math_issues[
            [
                "Reciept index",
                "raw_text",
                "quantity",
                "unit_price",
                "total_price",
                "calculated_amount",
                "amount_difference",
            ]
        ].to_string(index=False)
    )

# -----
# 8. PRODUCT SEQUENCE CHECK
# -----

print("\n" + "=" * 70)
print("7. PRODUCT SEQUENCE CHECK")
print("=" * 70)

sequence_issues = []

for receipt_id, group in product_df.groupby(product_receipt_id_col):

```

```

sequence = pd.to_numeric(
    group["sequence_order"],
    errors="coerce"
).dropna().astype(int).tolist()

expected = list(range(1, len(sequence) + 1))

if sorted(sequence) != expected:
    sequence_issues.append({
        "receipt_id": receipt_id,
        "products": len(group),
        "sequence_found": sequence,
        "expected": expected
    })

print(f"Receipts with sequence-order problems: {len(sequence_issues)}")

if sequence_issues:
    print(pd.DataFrame(sequence_issues).to_string(index=False))

# -----
# 9. RECEIPTS WITHOUT PRODUCTS
# -----

print("\n" + "=" * 70)
print("8. RECEIPT ↔ PRODUCT COVERAGE")
print("=" * 70)

receipt_id_set = set(receipt_ids)
product_receipt_id_set = set(product_receipt_ids)

receipts_without_products = sorted(
    receipt_id_set - product_receipt_id_set
)

products_per_receipt = (
    product_df.groupby(product_receipt_id_col)
    .size()
    .sort_values()
)

print(f"Receipts without product records: {len(receipts_without_products)}")

if receipts_without_products:
    print(receipts_without_products)

print("\nProduct count distribution:")
print(products_per_receipt.describe())

# -----
# 10. SUSPICIOUS NULL PRODUCT RECORDS
# -----

print("\n" + "=" * 70)
print("9. SUSPICIOUS PRODUCT RECORDS")
print("=" * 70)

```

```

# Products where the most important identifying fields are missing
suspicious_products = product_df[
    (
        product_df["item_name"].isna()
        | (product_df["item_name"].astype(str).str.strip() == "")
    )
    &
    (
        product_df["raw_text"].isna()
        | (product_df["raw_text"].astype(str).str.strip() == "")
    )
]

print(
    f"Products missing BOTH item_name and raw_text: "
    f"{len(suspicious_products)}"
)

if len(suspicious_products) > 0:
    print(suspicious_products.to_string(index=False))

# -----
# 11. RECEIPT TOTAL VS PRODUCT TOTAL
# -----

print("\n" + "=" * 70)
print("10. RECEIPT TOTAL vs PRODUCT TOTAL")
print("=" * 70)

receipt_totals = receipt_df.copy()
receipt_totals["total_amount_numeric"] = pd.to_numeric(
    receipt_totals["total_amount"],
    errors="coerce"
)

product_totals = product_df.copy()
product_totals["total_price_numeric"] = pd.to_numeric(
    product_totals["total_price"],
    errors="coerce"
)

product_sum = (
    product_totals.groupby(product_receipt_id_col)["total_price_numeric"]
    .sum(min_count=1)
)

comparison = receipt_totals[
    ["receipt_id", "total_amount_numeric"]
].copy()

comparison["product_total"] = comparison["receipt_id"].map(product_sum)

comparison["difference"] = (
    comparison["total_amount_numeric"]
    - comparison["product_total"]
).abs()

total_mismatch = comparison[
    comparison["total_amount_numeric"].notna()

```

```

        & comparison["product_total"].notna()
        & (comparison["difference"] > 0.02)
    ]

    print(
        f"Receipts where receipt total differs from sum of product totals: "
        f"{len(total_mismatch)}"
    )

    if len(total_mismatch) > 0:
        print("\nThese require MANUAL REVIEW:")
        print(
            total_mismatch[
                [
                    "receipt_id",
                    "total_amount_numeric",
                    "product_total",
                    "difference",
                ]
            ].to_string(index=False)
        )

# -----
# 12. FINAL SUMMARY
# -----

print("\n" + "=" * 70)
print("QUALITY CHECK SUMMARY")
print("=" * 70)

summary = {
    "Receipt records": len(receipt_df),
    "Product records": len(product_df),
    "Unique receipts": receipt_ids.nunique(),
    "Duplicate receipt IDs": len(duplicate_receipt_ids),
    "Missing receipt IDs": len(missing_receipt_ids),
    "Orphan product records": len(orphan_products),
    "Receipts without products": len(receipts_without_products),
    "Product price inconsistencies": len(product_math_issues),
    "Sequence-order problems": len(sequence_issues),
    "Products missing item_name + raw_text": len(suspicious_products),
    "Receipt/product total mismatches": len(total_mismatch),
}

summary_df = pd.DataFrame(
    list(summary.items()),
    columns=["Check", "Result"]
)

print(summary_df.to_string(index=False))

print("\n" + "=" * 70)
print("IMPORTANT")
print("=" * 70)
print(
    "The automated checks identify records that deserve manual review. "
    "They do NOT prove that an extraction is correct or incorrect."
)
print(

```

```
"Use the flagged receipt IDs to compare the extracted values against "  
"the actual receipt images."  
)  
print("=" * 70)
```

```
=====
```

QUALITY CHECK STARTED

```
=====
```

Receipt records: 124
Product records: 367

```
=====
```

1. STRUCTURE CHECK

```
=====
```

Receipt Details columns:

['receipt_id', 'receipt_number', 'receipt_date', 'receipt_time', 'subtot

Product Details columns:

['raw_text', 'item_name', 'quantity', 'unit_price', 'discount', 'total_p

```
=====
```

2. RECEIPT ID CHECK

```
=====
```

Unique receipt IDs: 124
Duplicate receipt IDs: 0
Missing receipt IDs: 0
Products linked to non-existent receipts: 0

```
=====
```

3. RECEIPT FIELD COMPLETENESS

```
=====
```

field	missing	missing_percentage
receipt_id	0	0.00
receipt_number	0	0.00
receipt_date	1	0.81
receipt_time	2	1.61
total_amount	0	0.00
currency	11	8.87
payment_method	2	1.61
image_url	0	0.00
ocr_raw_text	0	0.00

```
=====
```

4. PRODUCT FIELD COMPLETENESS

```
=====
```

field	missing	missing_percentage
raw_text	0	0.00
item_name	0	0.00
quantity	0	0.00
unit_price	0	0.00
discount	323	88.01
total_price	0	0.00
sequence_order	0	0.00
Reciept index	0	0.00

```
=====
5. NUMERIC FIELD CHECK
=====
Receipt numeric fields:
      field  non_numeric_values
      ^
```